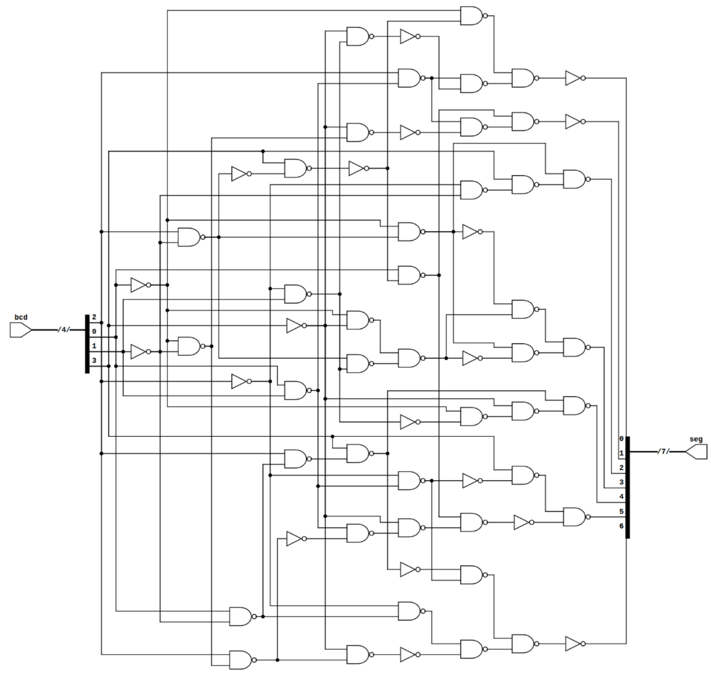
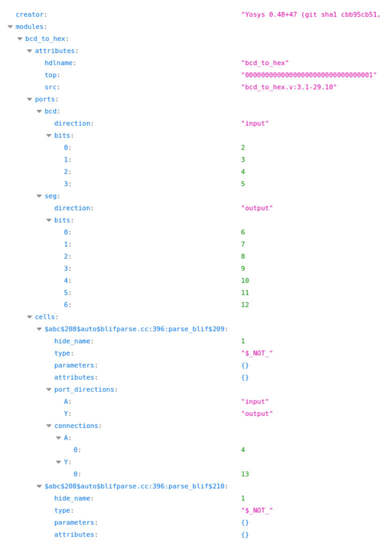
Create a Verilog file and validate it in AMD Vivado. A verilog module for a BCD to HEX decoder is shown to the right.

```verilog
`timescale 1ns / 1ps

module bcd_to_hex(

    input [3:0] bcd,
    output reg [6:0] seg);

    always @(*)
        begin
        case (bcd)
             0 : seg = 7'b1111110;
             1 : seg = 7'b0110000;
             2 : seg = 7'b1101101;
             3 : seg = 7'b1111001;
             4 : seg = 7'b0110011;
             5 : seg = 7'b1011011;
             6 : seg = 7'b1011111;
             7 : seg = 7'b1110000;
             8 : seg = 7'b1111111;
             9 : seg = 7'b1110011;
            10 : seg = 7'b1110111;
            11 : seg = 7'b0011111;
            12 : seg = 7'b1001110;
            13 : seg = 7'b0111101;
            14 : seg = 7'b1001111;
            15 : seg = 7'b1000111;
        endcase
        end
endmodule
```

Run Yosys logic optimizer on Verilog file. It outputs a JSON file that contains a minimized logic implementation of the verilog. The JSON file is then converted to an SVG image for viewing. An SVG schematic for the BCD to HEX decoder is shown to the right.

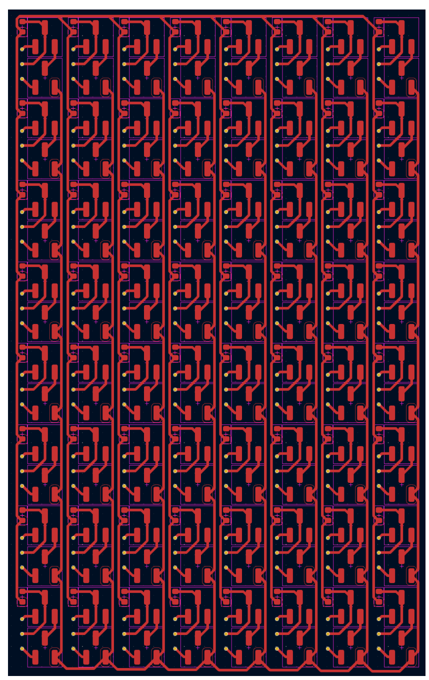


The JSON file is then parsed using the json package in Python. From there, all the information can be easily extracted. The net connections for each gate, input, and output are stored. A snippet of the JSON file is provided to the right.




```
creator:                "Yosys 0.48+47 (git sha1 cbb95cb51,
modules:
  bcd_to_hex:
    attributes:
      hdlname:          "bcd_to_hex"
      top:              "00000000000000000000000000000001"
      src:              "bcd_to_hex.v:3.1-29.10"
    ports:
      bcd:
        direction:      "input"
        bits:
          0:            2
          1:            3
          2:            4
          3:            5
      seg:
        direction:      "output"
        bits:
          0:            6
          1:            7
          2:            8
          3:            9
          4:            10
          5:            11
          6:            12
    cells:
      $abc$208$auto$blifparse.cc:396:parse_blif$209:
        hide_name:      1
        type:           "$_NOT_"
        parameters:     {}
        attributes:     {}
        port_directions:
          A:            "input"
          Y:            "output"
        connections:
          A:
            0:          4
          Y:
            0:          13
      $abc$208$auto$blifparse.cc:396:parse_blif$210:
        hide_name:      1
        type:           "$_NOT_"
        parameters:     {}
        attributes:     {}
```


These net connections can then be assigned to locations on a pre-arranged array of NAND gates. On the right is an example of what an 8x8 array of NAND gates looks like using 3-pin mosfets. The array could use IC's instead of mosfets.



A barebones Kicad6 file is generated. It will hold an array of NAND gates large enough to accomodate all logic gates needed in the design. The vias are then assigned net connections. Connectors for inputs and outputs are also generated. PCB is then autorouted.